

M4E WhatsApp CRM — Employee Operating Manual Review

Items 31–46: Comprehensive Analysis & Recommendations

Table of Contents

1. Executive Summary
 - Key Findings at a Glance
 - Immediate Action Required
2. Critical Issues
 - Item 31: Campaign Template Rate Display Bug
 - Item 40: Middleware Security Headers (Critical Gap)
3. Security Observations
 - Item 34: No Role Restrictions for AI Settings
 - Item 35: No Auto-Logout Timeout
 - Item 45: Rate Limiting Implementation
4. Code Quality Observations
 - Item 42: Error Handling Consistency
 - Item 43: Test Coverage
 - Item 44: Environment Variable Documentation
5. Feature Enhancement Observations
 - Item 32: Branch System Lacks Sub-Branch Hierarchy
 - Item 33: Employee Manual v3 Size Discrepancy
 - Item 36: Campaign AI Recommendations Basis
6. Research Findings & Strategic Recommendations
 - 6a. NCC SMS Sender ID Registration (Item 37)
 - 6b. Cal.com vs Calendly (Item 38)
 - 6c. NotebookLM Video System (Item 39)
7. Infrastructure & Operations
 - Item 41: Database Migration Numbering Gaps
 - Item 46: Backup and Disaster Recovery
8. Priority Implementation Matrix
 - Immediate (This Week)
 - Short-Term (Next 2 Weeks)
 - Medium-Term (Next Month)
 - Long-Term (Next Quarter)
9. Appendices
 - Appendix A: Files Referenced
 - Appendix B: Research Documents
 - Appendix C: Test File Inventory (31 Files)
 - Appendix D: API Routes Without Error Handling (21 Routes)
 - Appendix E: API Routes With Rate Limiting (16 Routes)

M4E WhatsApp CRM — Employee Operating Manual Review

Items 31-46: Comprehensive Analysis & Recommendations

Document Version: 1.0

Date: 1 July 2026

Prepared by: Marketing4Effect Technical Analysis Team

Platform: M4E Customer Reactivation Manager (WhatsApp CRM)

Codebase Snapshot: 471 TypeScript files | 99,000+ lines | 111
API routes | 53 migrations

Table of Contents

1. [Executive Summary](#)
 2. [Critical Issues](#)
 3. [Security Observations \(Items 34, 35, 40, 45\)](#)
 4. [Code Quality Observations \(Items 31, 42, 43, 44\)](#)
 5. [Feature Enhancement Observations \(Items 32, 33, 36\)](#)
 6. [Research Findings & Strategic Recommendations](#)
 - 6a. [NCC SMS Registration \(Item 37\)](#)
 - 6b. [Cal.com vs Calendly \(Item 38\)](#)
 - 6c. [NotebookLM Video System \(Item 39\)](#)
 7. [Infrastructure & Operations \(Items 41, 46\)](#)
 8. [Priority Implementation Matrix](#)
 9. [Appendices](#)
-

1. Executive Summary

This document addresses 16 observations (Items 31-46) identified during the Employee Operating Manual review of the M4E WhatsApp CRM platform. The observations span code bugs, security gaps, feature enhancement opportunities, strategic research findings, and infrastructure concerns.

Key Findings at a Glance

Category	Count	Critical	High	Medium	Low
Code Bugs	1	1	—	—	—
Security Gaps	4	1	2	1	—
Code Quality	3	—	2	1	—
Feature Enhancements	3	—	1	1	1
Research / Strategic	3	—	—	—	—
Infrastructure	2	—	1	1	—
Total	16	2	6	4	1

Immediate Action Required

1. **Fix the formatRate display bug** (Item 31) — rates display as “7850%” instead of “78.5%”. One-line fix.
2. **Add security headers to middleware** (Item 40) — the application is missing Content-Security-Policy, HSTS, X-Frame-Options, and other critical headers.
3. **Implement session timeout / auto-logout** (Item 35) — users remain logged in indefinitely, a significant risk in shared-computer environments common in Nigerian businesses.
4. **Add role-based access control to AI settings** (Item 34) — any authenticated user can currently modify AI model selection, token limits, and system prompts.

2. Critical Issues

These items require immediate attention due to their impact on data accuracy or security posture.

Item 31: Campaign Template Rate Display Bug

Field	Value
Status	Bug
Severity	Critical
Effort	Quick Fix (< 30 minutes)
File	src/components/campaigns/step2-template.tsx, line 163

Problem Description

The `formatRate` function in the campaign wizard's template selection step multiplies rates by 100 before displaying them. However, the database stores rates as whole-number percentages (e.g., 78.5 means 78.5%, not 0.785).

Current code (line 163):

```
const formatRate = (rate: number | null) => {  
  if (rate === null) return "-";  
  return `${Math.round(rate * 100)}%`;  
};
```

Database values from campaign templates (migration 041):

Template	Delivery Rate	Open Rate	Reply Rate	Conversion Rate
Win-Back	78.5	22.3	15.0	—
VIP Loyalty	85.0	28.5	22.0	—
Feedback Survey	92.0	35.0	28.0	—

Result: A delivery rate of 78.5 displays as “7850%” instead of “78.5%”.

Cross-Reference: `step6-review.tsx` is CORRECT

Importantly, `step6-review.tsx` handles rates correctly. At lines 278, 483, 496, 509, and 522, it divides rates by 100 for mathematical calculations (e.g., `expectedConversionRate / 100` to convert 78.5% to 0.785 for revenue estimation). This is the correct approach when the database stores whole-number percentages.

At line 581, it displays `{expectedConversionRate}%` conversion directly — also correct.

Recommended Fix

Option A — Simple rounding (matches step6 display pattern):

```
const formatRate = (rate: number | null) => {  
  if (rate === null) return "-";  
  return `${Math.round(rate)}%`;  
};
```

Option B — Preserve decimal precision:

```
const formatRate = (rate: number | null) => {
  if (rate === null) return "-";
  return `${rate.toFixed(1)}%`;
};
```

Recommendation: Use Option B for consistency with the database precision (one decimal place).

Verification Steps

1. Apply the fix to `src/components/campaigns/step2-template.tsx` line 163
 2. Navigate to Campaign Wizard → Step 2 (Template Selection)
 3. Verify rates display correctly (e.g., “78.5%” not “7850%”)
 4. Verify Step 6 (Review) still displays rates correctly
 5. Run `npm run build` to confirm no TypeScript errors
-

Item 40: Middleware Security Headers (Critical Gap)

Field	Value
Status	Security Gap
Severity	Critical
Effort	Quick Fix (< 1 hour)
File	<code>src/middleware.ts</code>

Problem Description

The middleware currently adds only operational headers (request ID via `x-request-id` and response timing via `x-response-time`) but is missing all standard security headers. This leaves the application vulnerable to:

- **Clickjacking** (no `X-Frame-Options` or `frame-ancestors` CSP)
- **XSS attacks** (no `Content-Security-Policy`)
- **Protocol downgrade attacks** (no `Strict-Transport-Security`)
- **MIME-type sniffing** (no `X-Content-Type-Options`)
- **Information leakage** (no `Referrer-Policy`, `Permissions-Policy`)

Recommended Fix

Add the following security headers to the middleware response:

```
// In src/middleware.ts, add to the response headers:
const securityHeaders = {
  // Prevent clickjacking
  'X-Frame-Options': 'DENY',
```

```

// Prevent XSS - adjust script-src as needed for your CDN/
// analytics
'Content-Security-Policy': [
  "default-src 'self'",
  "script-src 'self' 'unsafe-inline' 'unsafe-eval' https://
    vercel.live",
  "style-src 'self' 'unsafe-inline'",
  "img-src 'self' data: https: blob:",
  "font-src 'self' data:",
  "connect-src 'self' https://*.supabase.co wss://*.supabase.co
    https://openrouter.ai https://api.brevo.com",
  "frame-ancestors 'none'",
].join('; '),
// Force HTTPS (Vercel handles TLS, but this prevents downgrade)
'Strict-Transport-Security': 'max-age=31536000;
  includeSubDomains; preload',
// Prevent MIME-type sniffing
'X-Content-Type-Options': 'nosniff',
// Control referrer information
'Referrer-Policy': 'strict-origin-when-cross-origin',
// Restrict browser features
'Permissions-Policy':
  'camera=(), microphone=(), geolocation=(), payment=()',
// Prevent DNS prefetching abuse
'X-DNS-Prefetch-Control': 'on',
};

// Apply to response
Object.entries(securityHeaders).forEach(([key, value]) => {
  response.headers.set(key, value);
});

```

Important Notes

- The CSP connect-src must include all API endpoints the frontend calls (Supabase, OpenRouter, Brevo, etc.)
- If embedding the CRM in iframes is needed, change X-Frame-Options to SAMEORIGIN and frame-ancestors to 'self'
- Test thoroughly after adding CSP — overly restrictive policies can break functionality
- Consider using next.config.ts headers for static assets and middleware for dynamic routes

3. Security Observations

Item 34: No Role Restrictions for AI Settings

Field	Value
Status	Security Gap

Field	Value
Severity	High
Effort	Medium (2-4 hours)
Affected Routes	/ai-chatbot, /admin/ai-budget

Problem Description

The AI chatbot settings (model selection, token limits, prompt configuration, knowledge base management) and AI budget settings lack role-based access control. The CRM implements four roles — Owner, Admin, Agent, Viewer — with RLS policies at the database level, but the AI settings pages do not enforce role restrictions at the application layer.

Risk scenarios: - An Agent-level user could change the AI model from a cost-effective option to an expensive one, inflating costs - A Viewer could modify system prompts, potentially introducing harmful or off-brand responses - Any authenticated user could increase token limits, leading to unexpected API charges - Knowledge base modifications could corrupt the AI's response quality

Recommended Fix

1. **Page-level guards:** Add role checks to the AI settings page components:

```
// In the AI settings page component
import { useUserRole } from '@hooks/use-user-role';

export default function AISettingsPage() {
  const { role, isLoading } = useUserRole();

  if (isLoading) return <LoadingSkeleton />;
  if (role !== 'owner' && role !== 'admin') {
    return <AccessDenied message="Only Owners and Admins can
      modify AI settings." />;
  }
  // ... render settings
}
```

1. **API-level guards:** Add role verification to the corresponding API routes:

```
// In API route handlers for AI settings
const member = await getMemberWithRole(supabase, accountId,
  userId);
if (!member || ![ 'owner', 'admin' ].includes(member.role)) {
  return NextResponse.json(
    { error: 'Insufficient permissions' },
```

```
    { status: 403 }  
  );  
}
```

- 1. **Navigation hiding:** Conditionally show/hide AI settings menu items based on role
- 2. **Audit logging:** Log all AI settings changes with user ID, timestamp, and before/after values

Recommended Role Matrix for AI Settings

Setting	Owner	Admin	Agent	Viewer
View AI settings	Yes	Yes	No	No
Change AI model	Yes	Yes	No	No
Modify token limits	Yes	No	No	No
Edit system prompts	Yes	Yes	No	No
Manage knowledge base	Yes	Yes	No	No
View AI budget	Yes	Yes	Yes	No
Modify AI budget limits	Yes	No	No	No

Item 35: No Auto-Logout Timeout

Field	Value
Status	Security Gap
Severity	High
Effort	Medium (3-5 hours)
Affected Area	Session management (Supabase Auth)

Problem Description

The CRM has no session timeout or auto-logout mechanism. Once authenticated, users remain logged in indefinitely until they explicitly sign out or clear browser data. This is a significant security concern in Nigerian business environments where:

- **Shared office computers** are common in SMEs
- **Cybercafé usage** still occurs for business tasks
- **Staff turnover** may leave active sessions on former employees’ devices
- **NDPR compliance** requires reasonable session management controls

Recommended Implementation

Approach: Client-side idle detection + Supabase session refresh control

1. Create an idle detection hook:

```
// src/hooks/use-idle-timeout.ts
import { useEffect, useRef, useCallback } from 'react';
import { createClient } from '@lib/supabase/client';

const IDLE_TIMEOUT_MS = 30 * 60 * 1000; // 30 minutes
const WARNING_BEFORE_MS = 5 * 60 * 1000; // Warn 5 minutes before

export function useIdleTimeout() {
  const timeoutRef = useRef<NodeJS.Timeout>();
  const warningRef = useRef<NodeJS.Timeout>();

  const resetTimer = useCallback(() => {
    clearTimeout(timeoutRef.current);
    clearTimeout(warningRef.current);

    warningRef.current = setTimeout(() => {
      // Show "Session expiring soon" toast/modal
      showSessionWarning();
    }, IDLE_TIMEOUT_MS - WARNING_BEFORE_MS);

    timeoutRef.current = setTimeout(async () => {
      const supabase = createClient();
      await supabase.auth.signOut();
      window.location.href = '/login?reason=timeout';
    }, IDLE_TIMEOUT_MS);
  }, []);

  useEffect(() => {
    const events = ['mousedown', 'keydown', 'scroll', 'touchstart'];
    events.forEach(e => document.addEventListener(e, resetTimer));
    resetTimer();

    return () => {
      events.forEach(e => document.removeEventListener(e, resetTimer));
      clearTimeout(timeoutRef.current);
      clearTimeout(warningRef.current);
    };
  }, [resetTimer]);
}
```

1. Add to the root layout or auth provider:

```
// In the authenticated layout wrapper
import { useIdleTimeout } from '@/hooks/use-idle-timeout';

export function AuthenticatedLayout({ children }) {
  useIdleTimeout();
  return <>{children}</>;
}
```

- 1. **Configure Supabase session lifetime** in the Supabase dashboard:
 - Set JWT expiry to 1 hour (default is 3600 seconds)
 - Set refresh token rotation to enabled
 - Set refresh token reuse interval to 10 seconds
- 2. **Make timeout configurable per account** (optional enhancement):
 - Add session_timeout_minutes to account settings
 - Default: 30 minutes
 - Range: 15-480 minutes
 - Allow Owners to configure per their security policy

Recommended Timeout Defaults

Context	Timeout	Rationale
Standard session	30 minutes idle	Balance between security and usability
Admin/AI settings	15 minutes idle	Higher-risk pages need shorter timeout
Remember me (optional)	7 days	For trusted devices only, with re-auth for sensitive actions

Item 45: Rate Limiting Implementation

Field	Value
Status	Security Gap
Severity	Medium
Effort	Medium (4-6 hours)
File	src/lib/rate-limit.ts

Current State

A well-documented rate limiting module exists at src/lib/rate-limit.ts (5,232 bytes). It implements a fixed-window counter pattern with:

- Configurable limits and window sizes per key

- Opportunistic memory cleanup (every ~1,000 calls)
- Standard RateLimitResult interface with success, remaining, reset, and limit fields
- Clear documentation noting the single-instance limitation

However, coverage is incomplete:

Metric	Value
Total API routes	111
Routes WITH rate limiting	16 (14.4%)
Routes WITHOUT rate limiting	95 (85.6%)

Routes currently protected: - Account management (invitations, members, transfer-ownership) - WhatsApp operations (broadcast, send, webhook, templates, embedded-signup) - Health check, SMS test, admin monitoring

Notable unprotected routes (high risk): - campaigns/analyze/route.ts — triggers expensive database analysis RPC - campaigns/[id]/launch/route.ts — launches campaigns (could be abused) - automations/cron/route.ts — cron trigger endpoint - flows/cron/route.ts — flow cron trigger endpoint - onboarding/route.ts — onboarding endpoint - All campaign CRUD routes - All flow CRUD routes - All automation CRUD routes

Recommended Fix

1. Apply rate limiting to all public-facing and mutation routes:

```
// Create a middleware-level rate limiter or a reusable wrapper
import { checkRateLimit } from '@lib/rate-limit';

// Standard tiers
const RATE_LIMITS = {
  read: { limit: 60, windowMs: 60_000 }, // 60 req/min
  write: { limit: 20, windowMs: 60_000 }, // 20 req/min
  expensive: { limit: 5, windowMs: 60_000 }, // 5 req/min
    (analyze, launch)
  auth: { limit: 10, windowMs: 300_000 }, // 10 req/5min
  webhook: { limit: 200, windowMs: 60_000 }, // 200 req/min
    (WhatsApp callbacks)
};
```

1. Priority routes to protect immediately:

- campaigns/analyze (expensive DB operation)
- campaigns/[id]/launch (irreversible action)
- automations/cron and flows/cron (should be secret-protected too)
- onboarding (public-facing)

2. **Long-term:** Consider migrating to Redis/Upstash for rate limiting if the platform scales beyond a single instance (the current in-memory implementation explicitly documents this limitation).
-

4. Code Quality Observations

Item 42: Error Handling Consistency

Field	Value
Status	Enhancement
Severity	High
Effort	Large (8-16 hours)
Scope	111 API routes

Current State Analysis

Pattern	Count	Percentage
Routes with try/catch	90	81.1%
Routes WITHOUT try/catch	21	18.9%
Routes using NextResponse.json	111	100%
Routes with mixed response patterns	2	1.8%

Positive finding: All 111 routes consistently use NextResponse.json for responses, which is good.

Concern: 21 routes lack try/catch error handling, meaning unhandled exceptions will result in generic 500 errors without proper logging or user-friendly messages.

Routes without try/catch (21):

```
admin/monitoring/logs/route.ts
agent/events/route.ts
automations/[id]/duplicate/route.ts
automations/[id]/route.ts
automations/cron/route.ts
automations/route.ts
campaigns/[id]/launch/route.ts
campaigns/[id]/performance/route.ts
campaigns/[id]/route.ts
campaigns/analyze/route.ts
campaigns/route.ts
campaigns/templates/route.ts
flows/[id]/activate/route.ts
```

```
flows/[id]/route.ts
flows/[id]/runs/route.ts
flows/cron/route.ts
flows/route.ts
flows/templates/route.ts
invitations/[token]/peek/route.ts
invitations/[token]/redeem/route.ts
onboarding/route.ts
```

Two routes with mixed response patterns (using both `NextResponse.json` and `new Response()`): - `whatsapp/media/[mediaId]/route.ts` — likely returns binary media data - `whatsapp/webhook/route.ts` — likely returns plain text for Meta webhook verification

These mixed patterns are acceptable given their specific use cases.

Recommended Fix

1. Create a standardized error response utility:

```
// src/lib/api/error-response.ts
import { NextResponse } from 'next/server';

interface ApiError {
  error: string;
  code?: string;
  details?: unknown;
}

export function errorResponse(
  message: string,
  status: number = 500,
  code?: string,
  details?: unknown
): NextResponse<ApiError> {
  // Log server errors
  if (status >= 500) {
    console.error(`[API Error ${status}]`, { message, code, details });
  }
  return NextResponse.json(
    { error: message, ...(code && { code }), ...(details && { details }) },
    { status }
  );
}

export function withErrorHandling<T>(
  handler: (req: Request, ctx: T) => Promise<NextResponse>
) {
  return async (req: Request, ctx: T): Promise<NextResponse> => {
```

```

    try {
      return await handler(req, ctx);
    } catch (error) {
      console.error('[Unhandled API Error]', error);
      return ErrorResponse(
        'An unexpected error occurred',
        500,
        'INTERNAL_ERROR'
      );
    }
  };
}

```

1. **Wrap the 21 unprotected routes** with the `withErrorHandling` higher-order function
2. **Standardize error response shape** across all routes:

```

{
  "error": "Human-readable message",
  "code": "MACHINE_READABLE_CODE",
  "details": {}
}

```

Item 43: Test Coverage

Field	Value
Status	Enhancement
Severity	High
Effort	Large (ongoing)
Config	vitest.config.ts present

Current State (Corrected)

Contrary to the initial observation that “no automated tests exist,” the codebase contains **31 test files** covering critical library modules:

Category	Test Files	Coverage Area
WhatsApp API	11	Meta API, templates, webhooks, encryption, phone utils
Flows Engine	5	Edges, engine, fallback, layout, validation
Automations	2	Engine, validation
Contacts	2	Deduplication, CSV parsing

Category	Test Files	Coverage Area
Auth	2	Invitations, roles
Infrastructure	2	Rate limiting, broadcast status
Other	7	Currency, date utils, media upload, flow editor state

What IS covered: Core library modules — the business logic layer that handles WhatsApp API interactions, flow execution, automation processing, and data validation.

What is NOT covered: - API routes (0 of 111 routes have tests) - React components (0 of 158 components have tests) - Integration tests (end-to-end flows) - Database migration validation

Recommended Testing Strategy

Phase 1 — Protect critical paths (Week 1-2): - Add API route tests for: campaign launch, WhatsApp send, broadcast, authentication - Add integration tests for: campaign wizard flow, contact import, AI chatbot pipeline

Phase 2 — Expand coverage (Week 3-4): - Component tests for: campaign wizard steps, settings pages, dashboard widgets - Add database migration tests (apply/rollback verification)

Phase 3 — Continuous improvement (Ongoing): - Require tests for all new features (enforce via PR template) - Set up coverage reporting in CI/CD - Target: 60% line coverage within 3 months

Immediate action: Run existing tests to verify they pass:

```
npx vitest run
```

Item 44: Environment Variable Documentation

Field	Value
Status	Enhancement
Severity	Medium
Effort	Medium (2-3 hours)
File	.env.local.example (4,422 bytes)

Current State

A `.env.local.example` file exists and is reasonably comprehensive. It includes sections for: - Supabase configuration (URL, anon key, service role key) - WhatsApp Business API credentials - OpenRouter AI configuration - Brevo email/SMS API keys - Encryption keys - Automation secrets - Various feature flags

Gaps Identified

1. **No indication of which variables are required vs. optional** — a developer cannot tell which variables will cause the app to crash if missing
2. **No documentation of expected formats** — e.g., is `ENCRYPTION_KEY` base64? Hex? How many bytes?
3. **No validation at startup** — the app does not fail fast with clear error messages when required variables are missing
4. **No documentation of which variables are needed for which features** — e.g., which variables are only needed if SMS is enabled?

Recommended Fix

1. **Enhance `.env.local.example` with annotations:**

```
# =====  
# REQUIRED – App will not start without these  
# =====  
  
# Supabase (get from supabase.com dashboard > Settings > API)  
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co #  
    Required  
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ... #  
    Required  
SUPABASE_SERVICE_ROLE_KEY=eyJ... #  
    Required  
  
# =====  
# OPTIONAL – Features degrade gracefully  
# =====  
  
# AI Chatbot (required only if AI chatbot is enabled)  
OPENROUTER_API_KEY=sk-or-... # Optional: AI chatbot disabled  
    without this  
  
# SMS (required only if SMS campaigns are used)  
BREVO_API_KEY=xkeysib-... # Optional: SMS features disabled  
    without this
```

1. **Add startup validation** in `src/lib/env.ts`:


```

const REQUIRED_VARS = [
  'NEXT_PUBLIC_SUPABASE_URL',
  'NEXT_PUBLIC_SUPABASE_ANON_KEY',
  'SUPABASE_SERVICE_ROLE_KEY',
] as const;

export function validateEnv() {
  const missing = REQUIRED_VARS.filter(v => !process.env[v]);
  if (missing.length > 0) {
    throw new Error(
      `Missing required environment variables:\n$
      {missing.join('\n')}`
    );
  }
}

```

1. **Create a docs/ENVIRONMENT_VARIABLES.md** with a complete table of all variables, their purpose, format, default values, and which feature requires them.

5. Feature Enhancement Observations

Item 32: Branch System Lacks Sub-Branch Hierarchy

Field	Value
Status	Enhancement
Severity	Low
Effort	Large (16-24 hours)
Migration	036_multi_branch_support.sql

Current State

The branch schema is flat — branches belong to accounts but cannot have parent branches. The current schema supports:

```

CREATE TABLE branches (
  id uuid PRIMARY KEY,
  account_id uuid REFERENCES accounts(id),
  name text NOT NULL,
  -- ... other fields
  -- NO parent_branch_id column
);

```

Impact Assessment

For M4E's current target market (Nigerian mid-market businesses), the flat branch structure is **adequate for most use cases**:

- **Retail chains** with 5–50 locations: flat structure works fine
- **Service businesses** with multiple offices: flat structure works fine
- **Restaurants/hospitality** with multiple outlets: flat structure works fine

Where hierarchy would be needed: - **Banks** with Regional → District → Branch structures (e.g., GTBank, First Bank) - **Large retail chains** with Zone → Region → Store hierarchies (e.g., Shoprite Nigeria) - **Telecoms** with Regional → Area → Territory structures

Recommendation: Defer to Phase 3

This is a “nice to have” for the current market segment. If M4E acquires enterprise clients requiring hierarchical branches, implement as follows:

1. Add parent_branch_id column:

```
ALTER TABLE branches
  ADD COLUMN parent_branch_id uuid REFERENCES branches(id),
  ADD COLUMN depth integer DEFAULT 0,
  ADD COLUMN path text; -- Materialized path: '/root/region/
                        district/branch'
```

1. Add recursive CTE for hierarchy queries:

```
CREATE OR REPLACE FUNCTION get_branch_hierarchy(p_branch_id uuid)
RETURNS TABLE(id uuid, name text, depth integer, path text)
AS $$
  WITH RECURSIVE tree AS (
    SELECT id, name, 0 as depth, name::text as path
    FROM branches WHERE id = p_branch_id
    UNION ALL
    SELECT b.id, b.name, t.depth + 1, t.path || ' > ' || b.name
    FROM branches b JOIN tree t ON b.parent_branch_id = t.id
  )
  SELECT * FROM tree ORDER BY depth;
$$ LANGUAGE sql;
```

1. Update the branch management UI to show tree structure with expand/collapse

Item 33: Employee Manual v3 Size Discrepancy

Field	Value
Status	Enhancement
Severity	Medium
Effort	Large (8-16 hours)
Files	docs/ EMPLOYEE_OPERATING_GUIDE.md (v2), docs/ EMPLOYEE_OPERATING_MANUAL_v3.md (v3)

Size Comparison

Version	File Size	Word Count	Ratio
v2 (EMPLOYEE_OPERATING_GUIDE.md)	262,422 bytes	~39,000 words	100%
v3 (EMPLOYEE_OPERATING_MANUAL_v3.md)	69,715 bytes	~10,356 words	27%

Analysis

v3 was described as a “complete rewrite from codebase audit” — meaning it was generated by analyzing the actual source code rather than being a manual update of v2. This approach has strengths and weaknesses:

Strengths of v3: - Accurately reflects the current codebase state
- Eliminates outdated information from v2 - Consistent structure derived from actual code analysis - More maintainable (smaller, focused)

Potential losses from v2: - Onboarding procedures and step-by-step walkthroughs - Business context and “why” explanations - Troubleshooting guides and common error resolutions - Nigerian market-specific operational guidance - Client communication templates and scripts - Workflow diagrams and process maps - FAQ sections based on real user questions

Recommendation

1. **Perform a structured diff** between v2 and v3 to identify specific content gaps
2. **Create a “v4” that merges the best of both:**
 - v3’s accurate technical documentation as the foundation
 - v2’s operational procedures, troubleshooting guides, and business context as supplements

3. **Separate concerns:** Consider splitting into:
- TECHNICAL_REFERENCE.md — API routes, database schema, configuration (from v3)
 - OPERATIONS_GUIDE.md — Procedures, workflows, troubleshooting (from v2)
 - ONBOARDING_GUIDE.md — New employee setup, training checklist (from v2)

Item 36: Campaign AI Recommendations Basis

Field	Value
Status	Enhancement (Documentation)
Severity	Medium
Effort	Quick Fix (1-2 hours)
Function	analyze_database_for_reactivation RPC (migration 041)

How the Campaign Analysis Works

The campaign wizard's "Analyze" step calls the analyze_database_for_reactivation PostgreSQL RPC function. Here is how it works:

1. Input: Takes an account_id (the tenant's account)

2. Configurable Thresholds: Reads from product_score_settings table: - tier1_max (default: 90 days) — "Active" customer threshold - tier2_max (default: 180 days) — "At-risk" customer threshold - Beyond tier2 = "Dormant"

3. Segmentation Logic:

Segment	Definition	Default
Active	Last purchase within tier1_max days	< 90 days ago
At-Risk	Last purchase between tier1_max and tier2_max days	90-180 days ago
Dormant	Last purchase beyond tier2_max days	> 180 days ago

4. Data Analyzed: - Total contacts in the account - Contacts with purchase history (from purchase_history table) - Revenue metrics: total revenue, average purchase value - Dormant revenue potential (estimated from historical purchase patterns)

5. Output: Returns a JSON object containing: - Contact counts per segment (active, at-risk, dormant) - Revenue statistics (total, average purchase value) - Dormant revenue potential estimate - Recommended campaign type based on segment distribution

Recommendation

1. **Document the analysis methodology** in the Employee Operating Manual
 2. **Add tooltips in the campaign wizard UI** explaining what each metric means
 3. **Make thresholds visible and editable** in the campaign wizard (currently only configurable via `product_score_settings`)
 4. **Add a “How this works” expandable section** in the Analyze step of the wizard
-

6. Research Findings & Strategic Recommendations

6a. NCC SMS Sender ID Registration (Item 37)

Field	Value
Status	Research Finding
Research Size	76,566 characters (comprehensive)
Source	/research/ncc_sms_registration_research.md

Key Findings

1. Registration is Mandatory

All alphanumeric SMS sender IDs used for A2P (Application-to-Person) messaging in Nigeria must be registered with each major carrier (MTN, Airtel, Glo, 9mobile) under NCC oversight. This applies to: - Domestic businesses of all sizes - Financial institutions and government agencies - International companies sending SMS into Nigeria - Any entity using branded sender names (not phone numbers)

2. Agencies CAN Register on Behalf of Clients

This is a critical finding for M4E’s business model. The regulatory and operator documentation confirms that agencies and aggregators can register sender IDs on behalf of multiple clients, provided: - Each client has a unique sender ID (no sharing) - The

agency maintains proper authorization documentation - Each registration includes the end-client's business details - Generic sender names (e.g., "INFO", "ALERT") are strictly prohibited

3. DND Registry Compliance is Critical

Over 30 million Nigerian phone numbers are on the Do-Not-Disturb (DND) registry. Sending promotional SMS to DND-registered numbers without explicit consent is a violation that can result in: - Fines from NCC - Sender ID revocation - Carrier blacklisting

4. New IA2P Aggregator Licence (July 2025)

The NCC has introduced an International A2P (IA2P) Aggregator Licence: - Cost: ₦10,000,000 for 5 years - Required for: routing international A2P SMS into Nigerian networks - Effective: July 2025 - Impact on M4E: Only relevant if M4E routes SMS from international sources; domestic SMS via local aggregators (e.g., Brevo, Termii) is not affected

5. Registration Process

The typical process involves: 1. Prepare documentation (CAC certificate, business letter, sender ID justification) 2. Submit to each carrier individually (or via an aggregator who handles this) 3. Wait for approval (typically 5-15 business days per carrier) 4. Receive NOC (No Objection Certificate) from each carrier 5. Activate sender ID through the SMS aggregator

Strategic Recommendations for M4E

1. Position as a Managed SMS Compliance Service:

- Offer sender ID registration as part of the CRM onboarding package
- Handle the carrier registration process on behalf of clients
- Charge a setup fee (₦50,000-₦100,000) plus annual maintenance
- This becomes a value-added service and a competitive differentiator

2. Partner with a Licensed SMS Aggregator:

- Use Termii, Brevo, or Africa's Talking as the SMS delivery partner
- These aggregators handle carrier relationships and DND compliance
- M4E focuses on the campaign strategy and CRM integration layer

3. Build DND Compliance into the CRM:

- Add a DND check before sending SMS campaigns
- Integrate with the DND registry API (available from carriers)

- Automatically exclude DND numbers from promotional campaigns
- Allow transactional messages to DND numbers (with proper consent documentation)
- 4. **Do NOT pursue the IA2P Aggregator Licence** at this stage:
 - ₦10M is a significant investment for a startup
 - M4E's clients are domestic Nigerian businesses
 - Use existing licensed aggregators for SMS delivery
 - Revisit if M4E expands to serve international clients sending SMS into Nigeria
- 5. **Update the CRM SMS Configuration:**
 - Add a "Sender ID Registration Status" field per client account
 - Track registration status per carrier (MTN, Airtel, Glo, 9mobile)
 - Block SMS campaigns until sender ID is registered and approved
 - The existing `src/components/settings/sms-sender-id.tsx` component should be enhanced with registration status tracking

6b. Cal.com vs Calendly (Item 38)

Field	Value
Status	Research Finding
Research Size	87,373 characters (comprehensive)
Source	/research/calcom_vs_calendly_research.md

Comparison Summary

Dimension	Cal.com	Calendly
License	AGPLv3 (open source)	Proprietary SaaS
Self-hosting	Yes (Docker, Railway, etc.)	No
Free tier	Unlimited events, payments, workflows	1 event type, basic features
White-label	Full (remove Cal.com branding)	Limited (Enterprise only)
API access	Full REST + webhooks (free)	REST + webhooks (paid tiers)
Pricing (paid)	\$12/user/mo (Teams)	\$10/user/mo (Standard)

Dimension	Cal.com	Calendly
Integrations	50+ (growing)	100+ (mature)
CRM connectors	HubSpot, Salesforce, Pipedrive	Salesforce, HubSpot, 20+ CRMs
Payment collection	Stripe, PayPal	Stripe, PayPal
Nigerian payments	Possible via self-hosted + Paystack	Not natively supported
Data sovereignty	Full (self-hosted)	US-hosted (Calendly servers)
Round-robin	Yes (free)	Yes (paid)
Team scheduling	Yes (free)	Yes (paid)

Recommendation: Cal.com (Strong)

For M4E's specific use case as a multi-client agency, Cal.com is the clear winner:

1. Cost advantage for agency model: - Cal.com free tier covers most scheduling needs - No per-seat costs for client scheduling pages - Self-hosting eliminates recurring SaaS fees - Estimated TCO: ₦15,000-₦30,000/month (VPS hosting) vs. ₦50,000+/month (Calendly Teams for 5+ users)

2. White-label capability: - Remove Cal.com branding entirely - Use client's domain (e.g., book.clientname.com) - Match client's brand colors and styling - This is impossible with Calendly without Enterprise pricing (\$15,000+/year)

3. Data sovereignty: - Self-hosted = data stays in Nigeria or chosen region - NDPR compliance is simpler when data location is controlled - No dependency on US-based SaaS provider

4. CRM integration: - Open API enables deep integration with M4E's WhatsApp CRM - Webhooks can trigger WhatsApp follow-ups after bookings - Custom fields can pass data between Cal.com and the CRM - Paystack integration possible via custom payment app

5. Nigerian market fit: - Paystack/Flutterwave integration via self-hosted customization - Nigerian time zone and locale support - WhatsApp notification integration (via webhooks)

Implementation Plan

Phase	Timeline	Actions
	Setup	

Phase	Timeline	Actions
Phase 1 (Week 1)		Deploy Cal.com on existing VPS or Railway; configure admin account
Phase 2 (Week 2)	Branding	Apply M4E branding; create default event types (Discovery Call, Strategy Session, Onboarding)
Phase 3 (Week 3)	Integration	Connect Cal.com webhooks to M4E CRM; auto-create contacts from bookings
Phase 4 (Week 4)	Client rollout	Create client-specific booking pages; set up round-robin for M4E team
Phase 5 (Month 2)	Enhancement	Add Paystack payment collection; WhatsApp booking confirmations

Note on Calendly

Calendly remains a valid choice for: - Individual professionals who want zero setup - Sales teams deeply integrated with Salesforce - Organizations that prefer managed SaaS over self-hosting - The Nigerian connection (founder Tope Awotona) may resonate with some clients

The current M4E website uses a Calendly embed at /book. This should be migrated to Cal.com as part of the implementation plan.

6c. NotebookLM Video System (Item 39)

Field	Value
Status	Research Finding
Video	"NotebookLM: I Built a Video System That Never Resets (Free)" by Tool Drop
Source	

Field	Value
	/research/ youtube_Q2nTaCahHMQ_clean.md

System Overview

The Tool Drop video presents a reusable video production system built on Google’s NotebookLM and custom Gemini Gems:

6 NotebookLM Notebooks (Knowledge Base):

Notebook	Purpose	M4E Equivalent
1. Competitor Research	Analyze competitor videos for patterns	M4E market research files
2. Audience Psychology	Understand viewer motivations and pain points	Customer avatar documents
3. Hook + Retention	Proven hook formulas and retention techniques	Copywriting frameworks
4. Video Techniques	Cinematic and editing techniques	Video pipeline config
5. Visual Identity	Consistent brand look across videos	M4E brand guide
6. Tool Prompt Terms	Optimized prompts for AI tools	Pipeline prompt templates

2 Custom Gemini Gems (Processing Engines):

Gem	Function	M4E Equivalent
Story Engine	Generates scripts from topic + notebook context	Pipeline scriptwriter stage
Asset Generator	Creates image/video prompts from scripts	Pipeline visual_planner stage

Production Tools: - ElevenLabs for voiceover (M4E uses edge-tts, free) - Nano Banana 2 for free AI video assets (M4E uses Pexels + FAL.ai)

Relevance to M4E's Video Pipeline

M4E already has a working video pipeline at /video-pipeline/ that produces videos for ~\$0.06 each. The NotebookLM system offers complementary ideas:

Already implemented in M4E pipeline: - Script generation (via LLM) - Visual asset generation (via FAL.ai + Pexels) - Voiceover (via edge-tts) - Automated assembly (via ffmpeg)

New ideas from NotebookLM system:

- 1. Persistent Knowledge Notebooks:** M4E could create structured knowledge bases that persist across video productions:
 - Nigerian market research notebook
 - Client industry templates (retail, beauty, food service)
 - Proven hook formulas for Nigerian audiences
 - Visual identity guidelines per client
- 2. Two-Stage Generation:** The separation of “Story Engine” (script) and “Asset Generator” (visuals) mirrors M4E’s pipeline but with more structured context injection. M4E could enhance its scriptwriter stage to pull from persistent knowledge bases.
- 3. Competitor Analysis Integration:** Systematically analyzing competitor videos before scripting could improve M4E’s content quality. This could be automated:
 - Scrape competitor YouTube channels
 - Analyze top-performing videos (views, engagement)
 - Extract patterns (hooks, topics, formats)
 - Feed into script generation

Recommended Integration Points

Priority	Integration	Effort	Impact
High	Create client-specific knowledge base files that the pipeline reads	4 hours	Better brand consistency
Medium	Add competitor analysis stage	8 hours	More relevant content

Priority	Integration	Effort	Impact
Medium	before scriptwriting Build a hook/ retention formula library	4 hours	Higher engagement
Low	Evaluate Nano Banana 2 as free alternative to FAL.ai	2 hours	Cost reduction
Low	Consider NotebookLM for client research organization	1 hour	Better research workflow

7. Infrastructure & Operations

Item 41: Database Migration Numbering Gaps

Field	Value
Status	Enhancement
Severity	Medium
Effort	Quick Fix (1-2 hours)
Directory	supabase/migrations/

Migration Health Check Results

Metric	Value
Total migration files	53
Numbered migrations	52
Non-numbered files	1 (rollback_035_036_037.sql)
Number range	001 to 056
Expected (no gaps)	56 files
Gaps found	5 (numbers 24, 25, 26, 27, 28)
Duplicate numbers	1 (049 appears twice)

Gap Details

Missing numbers 24-28: These 5 consecutive missing numbers suggest either: - A batch of migrations was developed and then abandoned/rolled back - A renumbering occurred at some point - Migrations were consolidated into fewer files

Duplicate number 049: -

049_rls_policies_and_campaign_templates.sql (9,267 bytes) -
049b_campaign_templates.sql (16,087 bytes)

The 049b suffix indicates this was an intentional split — the developer created a follow-up migration rather than modifying the original. This is acceptable practice but should be documented.

Rollback file: - rollback_035_036_037.sql — a manual rollback script for migrations 035-037, indicating these migrations were problematic at some point.

Risk Assessment

The gaps and duplicates are **cosmetic issues** that do not affect functionality: - Supabase applies migrations in alphabetical/numerical order - The actual migration state is tracked in supabase_migrations.schema_migrations table - Gaps do not cause application errors

Recommendations

1. **Document the gaps** in a supabase/migrations/README.md:

```
# Migration Notes
```

```
## Numbering Gaps
```

- 024-028: Removed during schema consolidation (date)
- 049/049b: Split migration for RLS policies and campaign templates

```
## Rollback Scripts
```

- rollback_035_036_037.sql: Manual rollback for branch/pipeline/scoring migrations

1. **Do NOT renumber existing migrations** — this would break the migration history in any deployed environment
2. **Verify migration state** in production:

```
SELECT * FROM supabase_migrations.schema_migrations ORDER BY
version;
```

- 1. **Adopt a timestamp-based naming convention** for future migrations (e.g., 20260701120000_feature_name.sql) to prevent numbering conflicts

Item 46: Backup and Disaster Recovery

Field	Value
Status	Enhancement
Severity	High
Effort	Medium (4-8 hours)
Scope	Supabase, Vercel, GitHub

Current State

No documented backup strategy exists for: - **Supabase database** — contains all customer data, conversations, campaigns, and configurations - **Supabase storage** — contains uploaded media, knowledge base files, and attachments - **Vercel deployments** — application code and environment variables - **GitHub repository** — source code (inherently backed up via Git, but no formal policy)

Risk Assessment

Data Store	Built-in Protection	Gap
Supabase DB (Pro plan)	Daily backups, 7-day retention, PITR	No off-site backup; no tested restore procedure
Supabase DB (Free plan)	No automatic backups	Complete data loss risk
Supabase Storage	No automatic backups	Media files at risk
Vercel	Immutable deployments (rollback possible)	Environment variables not backed up
GitHub	Distributed Git (inherent redundancy)	Low risk, but no mirror

Recommended Backup Strategy

1. Database Backups (Priority: Critical)

```
# Automated daily backup script
#!/bin/bash
# backup_supabase.sh

DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="/backups/supabase"
DB_URL="postgresql://postgres:${DB_PASSWORD}@db.$
        {SUPABASE_PROJECT}.supabase.co:5432/postgres"

# Create backup
pg_dump "$DB_URL" --format=custom --file="$BACKUP_DIR/
        m4e_crm_$DATE.dump"

# Compress
gzip "$BACKUP_DIR/m4e_crm_$DATE.dump"

# Upload to cloud storage (e.g., Backblaze B2, AWS S3, or Google
        Cloud Storage)
rclone copy "$BACKUP_DIR/m4e_crm_$DATE.dump.gz" remote:m4e-
        backups/database/

# Retain last 30 days locally
find "$BACKUP_DIR" -name "*.dump.gz" -mtime +30 -delete
```

2. Storage Backups (Priority: High)

```
# Sync Supabase storage buckets
SUPABASE_URL="https://${PROJECT_ID}.supabase.co"
for BUCKET in media knowledge-base attachments; do
    # Use Supabase Storage API to list and download files
    # Or use the S3-compatible endpoint if available
    rclone sync "supabase:$BUCKET" "remote:m4e-backups/storage/
        $BUCKET/"
done
```

3. Environment Variable Backup (Priority: High)

```
# Export Vercel environment variables (encrypted)
vercel env pull .env.backup --environment=production
gpg --encrypt --recipient admin@marketing4effect.com .env.backup
rclone copy .env.backup.gpg remote:m4e-backups/config/
rm .env.backup .env.backup.gpg
```

4. GitHub Mirror (Priority: Low)

```
# Mirror to a secondary Git host (e.g., GitLab, Bitbucket)
git push --mirror git@gitlab.com:marketing4effect/m4e-whatsapp-
        crm.git
```

5. Backup Schedule:

Component	Frequency	Retention	Storage
Database (full)	Daily at 02:00 WAT	30 days	Cloud storage
Database (WAL/PITR)	Continuous (Supabase Pro)	7 days	Supabase
Storage buckets	Daily at 03:00 WAT	30 days	Cloud storage
Environment variables	On every change	90 days	Encrypted cloud storage
GitHub mirror	Daily at 04:00 WAT	Continuous	Secondary Git host

6. Disaster Recovery Testing: - Test database restore monthly (to a staging environment) - Document the restore procedure step-by-step - Measure RTO (Recovery Time Objective) — target: < 2 hours - Measure RPO (Recovery Point Objective) — target: < 24 hours

8. Priority Implementation Matrix

Immediate (This Week)

Item	Description	Severity	Effort	Owner
31	Fix formatRate display bug	Critical	30 min	Developer
40	Add security headers to middleware	Critical	1 hour	Developer
35	Implement session timeout	High	3-5 hours	Developer
34	Add role restrictions to AI settings	High	2-4 hours	Developer

Short-Term (Next 2 Weeks)

Item	Description	Severity	Effort	Owner
42	Standardize error handling (21 routes)	High	8-16 hours	Developer

Item	Description	Severity	Effort	Owner
45	Extend rate limiting to critical routes	Medium	4-6 hours	Developer
46	Implement backup strategy	High	4-8 hours	DevOps
44	Enhance environment variable documentation	Medium	2-3 hours	Developer

Medium-Term (Next Month)

Item	Description	Severity	Effort	Owner
43	Expand test coverage (API routes, components)	High	Ongoing	Developer
33	Merge Employee Manual v2 + v3 into v4	Medium	8-16 hours	Technical Writer
36	Document campaign analysis methodology	Medium	1-2 hours	Technical Writer
41	Document migration gaps; adopt timestamp naming	Medium	1-2 hours	Developer
38	Deploy Cal.com (Phase 1-3)	Medium	3 weeks	Developer

Long-Term (Next Quarter)

Item	Description	Severity	Effort	Owner
37	Launch managed SMS compliance service	Strategic	Ongoing	Business
39	Integrate NotebookLM concepts into video pipeline	Low	8-16 hours	Developer
32	Add sub-branch hierarchy (if	Low	16-24 hours	Developer

Item	Description	Severity	Effort	Owner
	enterprise clients acquired)			

9. Appendices

Appendix A: Files Referenced

File	Purpose
src/components/campaigns/step2-template.tsx	Campaign wizard template selection (Item 31 bug)
src/components/campaigns/step6-review.tsx	Campaign wizard review step (rate calculations)
src/middleware.ts	Next.js middleware (security headers)
src/lib/rate-limit.ts	Rate limiting module
supabase/migrations/ 036_multi_branch_support.sql	Branch schema
supabase/migrations/041_campaign_engine.sql	Campaign engine + analysis RPC
supabase/migrations/ 049_rls_policies_and_campaign_templates.sql	RLS policies
supabase/migrations/ 049b_campaign_templates.sql	Campaign templates
.env.local.example	Environment variable template
vitest.config.ts	Test configuration
docs/EMPLOYEE_OPERATING_GUIDE.md	Employee Manual v2
docs/EMPLOYEE_OPERATING_MANUAL_v3.md	Employee Manual v3

Appendix B: Research Documents

Document	Size	Topic
/research/ ncc_sms_registration_research.md	76,566 chars	NCC SMS Sender ID Registration

Document	Size	Topic
/research/ calcom_vs_calendly_research.md	87,373 chars	Cal.com vs Calendly Comparison
/research/ youtube_Q2nTaCahHMQ_clean.md	2,679 chars	NotebookLM Video System Analysis

Appendix C: Test File Inventory (31 Files)

Category	Files
WhatsApp API	meta-api.test.ts, meta-api.media.test.ts, meta-api.resumable.test.ts, template-components.test.ts, template-header-handle.test.ts, template-lifecycle.test.ts, template-send-builder.test.ts, template-status-normalize.test.ts, template-validators.test.ts, template-webhook.test.ts, webhook-signature.test.ts
Flows	edges.test.ts, engine.test.ts, fallback.test.ts, layout.test.ts, validate.test.ts
Automations	engine.test.ts, validate.test.ts
Contacts	dedupe.test.ts, parse-contact-csv.test.ts
Auth	invitations.test.ts, roles.test.ts
WhatsApp Utils	encryption.test.ts, phone-utils.test.ts, registration.test.ts
Infrastructure	rate-limit.test.ts, broadcast-status.test.ts
Other	currency.test.ts, date-utils.test.ts, upload-

Category	Files
	media.test.ts, flow-editor-state.test.ts

Appendix D: API Routes Without Error Handling (21 Routes)

admin/monitoring/logs
 agent/events
 automations/[id]/duplicate
 automations/[id]
 automations/cron
 automations
 campaigns/[id]/launch
 campaigns/[id]/performance
 campaigns/[id]
 campaigns/analyze
 campaigns
 campaigns/templates
 flows/[id]/activate
 flows/[id]
 flows/[id]/runs
 flows/cron
 flows
 flows/templates
 invitations/[token]/peek
 invitations/[token]/redeem
 onboarding

Appendix E: API Routes With Rate Limiting (16 Routes)

account/invitations/[id]
 account/invitations
 account/members/[userId]
 account
 account/transfer-ownership
 admin/monitoring
 health
 invitations/[token]/peek
 invitations/[token]/redeem
 sms/test
 whatsapp/broadcast
 whatsapp/embedded-signup/callback
 whatsapp/react
 whatsapp/send
 whatsapp/templates/submit
 whatsapp/webhook

End of Document

This analysis was prepared from direct codebase inspection, database schema review, and comprehensive research documents. All code references include specific file paths and line numbers for verification.